

Exercise 1: Inventory Management System

Product.java

```
public class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    @Override
    public String toString() {
        return productId + " " + productName +
            " Qty:" + quantity + " Price:" + price;
    }
}
```

InventoryManagement.java

```
import java.util.HashMap;

public class InventoryManagement {

    static HashMap<Integer, Product> inventory = new HashMap<>();

    public static void addProduct(Product p) {
        inventory.put(p.productId, p);
    }

    public static void updateProduct(int id, int quantity) {
        if (inventory.containsKey(id)) {
            inventory.get(id).quantity = quantity;
        }
    }

    public static void deleteProduct(int id) {
```

```

        inventory.remove(id);
    }

    public static void display() {
        for (Product p : inventory.values()) {
            System.out.println(p);
        }
    }

    public static void main(String[] args) {
        addProduct(new Product(101, "Laptop", 10, 50000));
        addProduct(new Product(102, "Mouse", 50, 500));

        display();

        updateProduct(101, 15);
        deleteProduct(102);

        System.out.println("\nAfter Update/Delete");
        display();
    }
}

```

Complexity

- Add $\rightarrow O(1)$
- Update $\rightarrow O(1)$
- Delete $\rightarrow O(1)$

Exercise 2: E-Commerce Search Function

Product.java

```

public class Product {
    int productId;
    String productName;
    String category;

    public Product(int id, String name, String category) {
        this.productId = id;
        this.productName = name;
        this.category = category;
    }
}

```

```
}  
}
```

SearchDemo.java

```
public class SearchDemo {  
  
    public static int linearSearch(Product[] products, String key) {  
        for (int i = 0; i < products.length; i++) {  
            if (products[i].productName.equalsIgnoreCase(key))  
                return i;  
        }  
        return -1;  
    }  
  
    public static int binarySearch(Product[] products, String key) {  
        int low = 0, high = products.length - 1;  
  
        while (low <= high) {  
            int mid = (low + high) / 2;  
            int result = products[mid].productName.compareToIgnoreCase(key);  
  
            if (result == 0)  
                return mid;  
            else if (result < 0)  
                low = mid + 1;  
            else  
                high = mid - 1;  
        }  
        return -1;  
    }  
  
    public static void main(String[] args) {  
  
        Product[] products = {  
            new Product(1,"Keyboard","Electronics"),  
            new Product(2,"Laptop","Electronics"),  
            new Product(3,"Mouse","Electronics")  
        };  
  
        System.out.println(  
            "Linear Search: " +
```

```
        linearSearch(products,"Laptop"));

    System.out.println(
        "Binary Search: " +
        binarySearch(products,"Laptop"));
    }
}
```

Complexity

- Linear Search $\rightarrow O(n)$
- Binary Search $\rightarrow O(\log n)$

Exercise 3: Sorting Customer Orders

Order.java

```
public class Order {
    int orderId;
    String customerName;
    double totalPrice;

    public Order(int id, String name, double price) {
        this.orderId = id;
        this.customerName = name;
        this.totalPrice = price;
    }

    public String toString() {
        return orderId + " " + customerName +
            " " + totalPrice;
    }
}
```

SortOrders.java

```
public class SortOrders {

    static void bubbleSort(Order[] arr) {
        int n = arr.length;

        for(int i=0;i<n-1;i++) {
            for(int j=0;j<n-i-1;j++) {
                if(arr[j].totalPrice > arr[j+1].totalPrice) {
```

```

        Order temp = arr[j];
        arr[j]=arr[j+1];
        arr[j+1]=temp;
    }
}
}
}

```

```

static int partition(Order[] arr,int low,int high) {
    double pivot = arr[high].totalPrice;
    int i = low-1;

    for(int j=low;j<high;j++) {
        if(arr[j].totalPrice < pivot) {
            i++;
            Order temp=arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
        }
    }

    Order temp=arr[i+1];
    arr[i+1]=arr[high];
    arr[high]=temp;

    return i+1;
}

```

```

static void quickSort(Order[] arr,int low,int high) {
    if(low<high) {
        int pi=partition(arr,low,high);
        quickSort(arr,low,pi-1);
        quickSort(arr,pi+1,high);
    }
}

```

```

public static void main(String[] args) {

    Order[] orders = {
        new Order(1,"Ram",3000),
        new Order(2,"John",1000),

```

```

        new Order(3,"Sam",5000)
    };

    quickSort(orders,0,orders.length-1);

    for(Order o:orders)
        System.out.println(o);
    }
}

```

Complexity

- Bubble Sort $\rightarrow O(n^2)$
- Quick Sort $\rightarrow O(n \log n)$

Exercise 4: Employee Management System

Employee.java

```

public class Employee {
    int employeeId;
    String name;
    String position;
    double salary;

    public Employee(int id,String name,
        String position,double salary) {
        this.employeeId=id;
        this.name=name;
        this.position=position;
        this.salary=salary;
    }

    public String toString() {
        return employeeId+" "+name+
            " "+position+" "+salary;
    }
}

```

EmployeeManagement.java

```

public class EmployeeManagement {

    Employee[] employees = new Employee[10];
}

```

```
int count = 0;

void add(Employee e) {
    employees[count++] = e;
}

void search(int id) {
    for(int i=0;i<count;i++) {
        if(employees[i].employeeId==id)
            System.out.println(employees[i]);
    }
}

void traverse() {
    for(int i=0;i<count;i++)
        System.out.println(employees[i]);
}

void delete(int id) {
    for(int i=0;i<count;i++) {
        if(employees[i].employeeId==id) {
            for(int j=i;j<count-1;j++)
                employees[j]=employees[j+1];
            count--;
        }
    }
}

public static void main(String[] args) {
    EmployeeManagement obj =
        new EmployeeManagement();

    obj.add(new Employee(1,"Arun","Manager",50000));
    obj.add(new Employee(2,"Kumar","Developer",40000));

    obj.traverse();

    obj.delete(1);

    System.out.println("\nAfter Delete");
    obj.traverse();
}
```

```
}  
}
```

Exercise 5: Task Management System

Task.java

```
public class Task {  
    int taskId;  
    String taskName;  
    String status;  
  
    Task next;  
  
    public Task(int id,String name,String status) {  
        this.taskId=id;  
        this.taskName=name;  
        this.status=status;  
    }  
}
```

TaskManagement.java

```
public class TaskManagement {  
  
    Task head;  
  
    void add(int id,String name,String status) {  
        Task t = new Task(id,name,status);  
  
        if(head==null)  
            head=t;  
        else {  
            Task temp=head;  
            while(temp.next!=null)  
                temp=temp.next;  
            temp.next=t;  
        }  
    }  
  
    void display() {  
        Task temp=head;
```



```

while(temp!=null) {
    System.out.println(
        temp.taskId+" "+
        temp.taskName+" "+
        temp.status);
    temp=temp.next;
}
}

void delete(int id) {
    if(head.taskId==id) {
        head=head.next;
        return;
    }

    Task temp=head;
    while(temp.next!=null &&
        temp.next.taskId!=id)
        temp=temp.next;

    if(temp.next!=null)
        temp.next=temp.next.next;
}

public static void main(String[] args) {

    TaskManagement tm =
        new TaskManagement();

    tm.add(1,"Coding","Pending");
    tm.add(2,"Testing","Completed");

    tm.display();

    tm.delete(1);

    System.out.println("\nAfter Delete");
    tm.display();
}
}

```

Exercise 6: Library Management System

Book.java

```
public class Book {  
    int bookId;  
    String title;  
    String author;  
  
    public Book(int id,String title,String author) {  
        this.bookId=id;  
        this.title=title;  
        this.author=author;  
    }  
}
```

LibrarySearch.java

```
public class LibrarySearch {  
  
    static int linearSearch(Book[] books,  
        String title) {  
  
        for(int i=0;i<books.length;i++) {  
            if(books[i].title.equalsIgnoreCase(title))  
                return i;  
        }  
        return -1;  
    }  
  
    static int binarySearch(Book[] books,  
        String title) {  
  
        int low=0,high=books.length-1;  
  
        while(low<=high) {  
            int mid=(low+high)/2;  
  
            int result=  
                books[mid].title.compareToIgnoreCase(title);  
  
            if(result==0)  
                return mid;  
        }  
    }  
}
```

```

        else if(result<0)
            low=mid+1;
        else
            high=mid-1;
    }
    return -1;
}

public static void main(String[] args) {

    Book[] books = {
        new Book(1,"C","Author1"),
        new Book(2,"Java","Author2"),
        new Book(3,"Python","Author3")
    };

    System.out.println(
        linearSearch(books,"Java"));

    System.out.println(
        binarySearch(books,"Java"));
}
}

```

Exercise 7: Financial Forecasting

FinancialForecast.java

```

public class FinancialForecast {

    static double futureValue(double amount,
                               double rate,
                               int years) {

        if(years==0)
            return amount;

        return futureValue(
            amount*(1+rate),
            rate,
            years-1);
    }
}

```

```
public static void main(String[] args) {  
  
    double result =  
        futureValue(10000,0.10,5);  
  
    System.out.println(  
        "Future Value = " + result);  
}  
}
```

Complexity

- Recursive approach $\rightarrow O(n)$
- Space Complexity $\rightarrow O(n)$

- DSA_Exercises
 - JRE System Library [jdk-24](1)
 - src
 - exercise1
 - InventoryManagement.java
 - Product.java
 - exercise2
 - Product.java
 - SearchDemo.java
 - exercise3
 - Order.java
 - SortOrders.java
 - exercise4
 - Employee.java
 - EmployeeManagement.java
 - exercise5
 - Task.java
 - TaskManagement.java
 - exercise6
 - Book.java
 - LibrarySearch.java
 - exercise7
 - FinancialForecast.java

Eclipse IDE workspace: DSA_Exercises/src/exercise1/InventoryManagement.java - Eclipse IDE

Package Explorer: DSA_Exercises > JRE System Library [jdk-24](1) > src > exercise1 > InventoryManagement.java

```
1 package exercise1;
2
3 import java.util.HashMap;
4
5 public class InventoryManagement {
6
7     static HashMap<Integer, Product> inventory = new HashMap<>();
8
9     public static void addProduct(Product p) {
10         inventory.put(p.productId, p);
11     }
12
13     public static void updateProduct(int id, int quantity) {
14         if (inventory.containsKey(id)) {
15             inventory.get(id).quantity = quantity;
16         }
17     }
18
19     public static void deleteProduct(int id) {
20         inventory.remove(id);
21     }
22
23     public static void display() {
24         for (Product p : inventory.values()) {
25             System.out.println(p);
26         }
27     }
28
29     public static void main(String[] args) {
30         addProduct(new Product(101, "Laptop", 10, 50000));
31         addProduct(new Product(102, "Mouse", 50, 500));
32     }
33 }
```

Problems: Javadoc Declaration Console

Console:

```
<terminated> InventoryManagement [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (3 Jul 2026, 8:28:39 pm - 8:28:40 pm) [pid: 1]
Initial Inventory:
101 Laptop Qty:10 Price:50000.0
102 Mouse Qty:50 Price:500.0

After Update/Delete:
101 Laptop Qty:15 Price:50000.0
```

Task List: Find Friday - Today, Saturday, This Week, Next Sunday, Next Monday, Next Tuesday, Next Wednesday, Next Thursday, Next Friday, Next Saturday, Next Week, Two Weeks, Future, Outgoing, Incoming, Unscheduled, Completed

Outline: exercise1 > InventoryManagement > inventory: HashMap<Integer, Product> > addProduct(Product): void > updateProduct(int, int): void > deleteProduct(int): void > display(): void > main(String[]): void

Boot Dashboard: Type tags, projects, or working set names to match (incl. * and ? wildcards)

exercise1.InventoryManagement.java - DSA_Exercises/src











